Making AI Happen in Testing Projects

SQF 2026 — One-Day Hands-On Tutorial (Milan)

Presenter: Abbas Ahmad, UAESTQB **Audience:** Test engineers, test architects, and QA leads (no prior AI or strong Python background required) **Format:** One day — short theory, lots of hands-on lab

---

About This Workshop

AI is becoming part of everyday software engineering. This workshop shows how to use it — first the **cloud LLM APIs** (OpenAI and Claude) to learn the building blocks, then **locally** on your own laptop for **data privacy, cost control, and transparency**.

The local core runs **on your own laptop** in plain Python — no SaaS tools, no no-code platforms, and your documents never leave your machine. By the end of the day you will have built a small **local AI assistant for testing** that retrieves answers from your own requirement documents.

What You Will Build

A simple, readable pipeline you build up step by step:

1. **See tokenization** in the browser — how text becomes tokens.
2. **Use the cloud LLM APIs** — call **OpenAI (GPT)** and **Anthropic (Claude)** from Python to learn the building blocks: tokens, prompts, system messages, temperature, reasoning models, and multi-turn chat.
3. **A local LLM** — run **Llama 3B** on your laptop with **Ollama** and ask it an ISTQB question. It answers from memory — and may get it wrong.
4. **RAG retrieval from scratch** — pull the ISTQB syllabus PDF, chunk it, embed it with a small local model, and find the most relevant pieces using **cosine similarity**. No LLM, no API.
5. **RAG + LLM together** — feed the retrieved syllabus chunks to the local model so it finally answers the same question **correctly and grounded** in the document.

Prerequisites

Please prepare your laptop **before** the workshop. This section lists the hardware you need, the software to install, and **every command to verify each piece is working**. Copy-paste the commands; the *check* commands confirm a piece is installed before you move on.

> **Windows vs macOS/Linux.** On Windows the command is usually `python` (and sometimes `py`). On macOS/Linux it is often `python3` and `pip3`. Wherever you see `python` below, use whichever one works on your machine — see Install Python for how to find the right one.

Hardware Requirements

Resource	Requirement	Notes
RAM	**16 GB minimum**	Recommended for smooth performance with local AI models
Storage	**~10 GB free**	~2 GB Python environment & dependencies · ~2 GB local Llama 3B model · remainder for the embedding model and working files
Admin rights	**Required**	You must be able to install Python packages via `pip` and install Ollama

Software Checklist

Software	Purpose	Download
Python (latest)	Runs the tutorial code	https://www.python.org
Visual Studio Code	Recommended editor (free)	https://code.visualstudio.com
Ollama	Runs the local Llama model (free)	https://ollama.com/download

---

1. Install Python

Install the **latest version of Python** from https://www.python.org, then **reopen your terminal** so it picks up the new install.

Verify Python is installed — run:

```
python --version
```

You should see a version number, e.g. `Python 3.12.4`.

If `python` does not work (command not found, or it does nothing), try these alternatives — one of them will work depending on your operating system:

```
python3 --version    # macOS / Linux usually
py --version         # Windows launcher
```

Use whichever command returned a version number for **every** `python` command in this guide.

Verify pip is installed (pip is Python's package installer — it ships with Python):

```
pip --version
```

`pip -V` does the same thing. If `pip` is not found, try `pip3 --version` or `py -m pip --version`.

---

2. Install Visual Studio Code

Download and install **Visual Studio Code** (free): https://code.visualstudio.com

It is the recommended editor for the workshop. Any IDE you are comfortable with also works.

---

3. Set Up the Project (virtual environment + packages)

A **virtual environment** ("venv") is a private folder of packages just for this project, so we don't touch your system Python. We use **one environment for the whole workshop** — both the `tutorial/` part and the `GenAI_Intro/` part — so you set it up **once** and never have to switch. Run these steps from the **repo root** (the folder this README is in).

Step 1 — create and activate the virtual environment:

```
python -m venv venv
```

Then activate it:

```
source venv/bin/activate     # macOS / Linux
venv\Scripts\activate        # Windows
```

After activating, your prompt shows `(venv)`. Confirm it is using the venv's Python:

```
python --version
```

Step 2 — install all the workshop requirements (one file at the root):

```
pip install -r requirements.txt
```

Verify the packages installed:

```
pip list
```

You should see `sentence-transformers`, `numpy`, `pypdf`, `ollama`, `Flask`, `openai`, and `anthropic` in the list.

> **Leaving / switching environments:** to exit a venv, type `deactivate` (same command on macOS, Linux, and Windows). Because this one environment covers everything, you won't need to — activate it once and use it for both parts of the workshop. Just re-run the activate command above if you open a new terminal.

4. Install Ollama (the local LLM runner)

1. Download and install **Ollama** (free): https://ollama.com/download (macOS / Windows: run the installer; it then runs in the background.)
2. **Verify Ollama is installed:**

```
ollama --version
```

3. **Download the model** (~2 GB, one time only):

```
ollama pull llama3.2:3b
```

4. **Check which models you have downloaded** — this command **lists the different models that were downloaded** to your machine:

```
ollama list
```

`llama3.2:3b` should appear in the list.

5. **Test that the model works** — this command **runs the model** and asks it a question (no Python needed):

```
ollama run llama3.2:3b "Say hello in one sentence."
```

If it prints a sentence, your local model is running. Type `/bye` to leave the chat.

The Hands-On Lab

The lab runs in **five steps**, across two folders:

```
GenAI_Intro/            # Step 1 — the cloud-API lab (OpenAI + Claude)
├── todo/               #   learner version: implement each route yourself
├── solution/           #   completed version to compare against
└── exercise.md         #   step-by-step walkthrough

tutorial/               # Steps 2–4 — local LLM + RAG (no cloud, no API
keys)
├── 1_local_llm.py      #   run + call a local Llama 3B (via Ollama)
├── 2_rag.py            #   RAG retrieval from the syllabus PDF, no AI
model
```

```
├── 3_rag_plus_llm.py   #   combine retrieval with the local model
├── docs/               #   the ISTQB CTFL syllabus PDF (add your own
.pdf/.md)
└── README.md           #   step-by-step instructions
```

> Packages for both parts of the workshop are installed once from the `requirements.txt` at the repo root — there is no per-folder requirements file. Keep the workshop virtual environment active (`(venv)` in your prompt) for every step; if it isn't, re-run the activate command from Step 1 of the setup.

The local steps (2 and 4) deliberately ask the **same question** ("what are the seven testing principles?"). In **Step 2** the model answers from memory and gets it **wrong**; in **Step 4**, fed the real syllabus, it answers **correctly**. That contrast is the whole point.

Step 0 — See tokenization in your browser (no code)

Open the OpenAI tokenizer and type a sentence: **https://platform.openai.com/tokenizer** (or https://tiktokenizer.vercel.app). Watch your text get split into **tokens** (small pieces) — the first thing every model does with text.

Step 1 — Intro to GenAI APIs (OpenAI + Claude)

Work through `GenAI_Intro/`: a small Flask app where you call the **OpenAI (GPT)** and **Anthropic (Claude)** APIs one route at a time — counting tokens, sending prompts, system messages, temperature, reasoning models, image generation, and a stateless multi-turn chat. Implement each route in `todo/`, compare with `solution/`.

👉 Full walkthrough: **GenAI_Intro/exercise.md**

> The rest of the steps run **locally** with no cloud and no API keys. Run them from inside `tutorial/` (`cd tutorial`).

Step 2 — Run and call a local LLM

```
python 1_local_llm.py
```

Asks the local Llama 3B model: *"what are the seven ISTQB testing principles?"* The model answers from its **general memory** — it has not seen the syllabus, so it confidently makes up the wrong principles. That's the problem RAG solves.

Step 3 — RAG retrieval, no AI model

```
python 2_rag.py
```

Reads the syllabus PDF in `docs/`, cuts it into chunks, turns each into a vector with a small local model, saves them to `embeddings.json`, and searches for the **same topic** using **cosine similarity** — no LLM,

no API. The first run is slower (it embeds the whole PDF once); later runs reuse `embeddings.json`. To force a rebuild, delete `embeddings.json`.

Step 4 — RAG + local LLM together

```
python 3_rag_plus_llm.py
```

Retrieves the relevant chunks from the syllabus, pastes them into the prompt as context, and asks the local model the **same question** — now answered **using only the syllabus**. Compare this answer to Step 2: this is why Retrieval-Augmented Generation matters.

> 👉 Full step-by-step local-lab instructions: **tutorial/README.md**

Using Your Own Documents

Drop any `.pdf` (or `.md`) file into `tutorial/docs/`, delete `embeddings.json` so it rebuilds, and re-run — the code picks up every PDF/Markdown file automatically.

What You Will Take Home

- A working **local RAG assistant** for test engineering you fully understand
- Clear, reusable patterns you can apply to your own projects
- A practical sense of **how to control and evaluate** AI in testing